HOT WOK, SMART CHIPS
We're Coooking!

Fengwei Fried Rice presents

A cook-along, play-along study guide to Hardware, Networks & Behaviourist AI — taught by cooking.

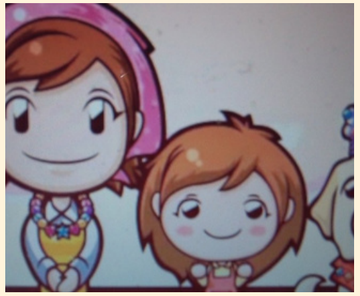


Stage 2 · Study Guide V1.0 · November 2026 · Introduction to Artificial Intelligence



The Kitchen Brigade

Team Fengwei Fried Rice · Module: Hardware, Networks & Behaviourist AI



Who cooked this guide?

纪小佩 — Head Chef (CEO)
林元甲 — Pantry Keeper (CIO)
温俊强 — Recipe Tester (Content Lead)
汤丽嘉 — Recipe Writer (Content Lead)
曾富良 — Plating Artist (Visual Design Lead)
李秀春 — Camera Crew (Video Lead)
黄施杰 — Taste Inspector (Quality Lead)

Version

V1.0 · first complete version for Stage 2

Source recipe book

Shared Markdown Content V2.0 · Main/Chapter-01 … 05

Kitchen address

github.com/Bvmhe/CS-AI-Group

Stage 2 · Study Guide V1.0 · November 2026 · Built from Shared Markdown Content V2.0 (Main/Chapter-01 … 05)

Today's Menu



Pick a dish — 01 → 05 is the chef-recommended route. Page numbers are on the right.

Main Courses

Recipe 01 · The Kitchen Crew — AI Hardware p.8

Recipe 02 · Order Up! — Networks & AI Infrastructure p.26

Recipe 03 · Training the Kitchen Pup — Behaviourist AI & RL p.44

Recipe 04 · Chef's Senses — AI Sensors & Intelligent Systems p.62

Recipe 05 · Running the Restaurant Chain — Integration & Optimization p.78





Starters & Desserts

Starter · How to Play p.4

Starter · Learning Goals p.5

Starter · Score Card p.6

Starter · Meet the Kitchen p.7

Dessert · Mama's Pantry (glossary) · Word Soup · Videos · References · Credits p.94

Treat · The Recipe Book (unlockable dishes) p.104



How to Play



Every page is a game screen. Here is how a round works.

1 · PICK

Choose a Recipe from the Menu. 01 → 05 is best: each one builds on the last.



2 · COOK ALONG

Each Step: Mama's Problem → Predict → Try it → Look → Why it works → Use it → Taste Test.



3 · PLAY

Mini-games have six parts: Goal · Prep · Task · Observe · Reflect · Mama's Feedback. Try before you peek!



4 · STAMP

Score Card: Gold = right first time · Silver = right after a hint · Bronze = read the answer, then redo it.



5 · UNLOCK

Finish a Recipe → unlock its page in the Recipe Book at the back!



Kitchen tools you need

Pen, paper and a calculator. A phone or laptop with internet for the 🌐 Kitchen Lab activities — every lab has an offline back-up (sample output + questions), so you can always finish.

What You Will Be Able to Cook

Learning goals, who this is for, and what to know first



LEARNING GOALS

1. Name CPU, GPU, TPU/NPU and memory, and explain what each does for AI. (Recipe 01)
2. Explain how networks move AI data, and choose cloud vs edge using latency and bandwidth. (Recipe 02)
3. Describe how an agent learns from rewards, and explain exploration vs exploitation. (Recipe 03)
4. Compare cameras, LiDAR and ultrasonic sensors, and explain sensor fusion. (Recipe 04)
5. Explain how hardware, networks and software are combined and optimised at scale. (Recipe 05)





BEFORE YOU START

For 1st/2nd-year CS & IT students.
You need: basic arithmetic, what a variable and a loop are, a rough idea of what a CPU and the internet are.
No electronics or advanced maths.
Time: about 1.5–2 hours per Recipe.



My Score Card



Stamp one medal per mini-game: 🥇 Gold · 🥈 Silver · 🥉 Bronze

1

RECIPE 01

1.1 Stir-Fry Maths ◯
1.2 Hire the Chef ◯
1.3 Conveyor Belt ◯
1.4 Chopping Race ◯
1.5 Pantry Run ◯
1.6 Batch Cooking ◯

2

RECIPE 02

2.1 Bento Packing ◯
2.2 Careful vs Shouting Waiter ◯
2.3 Delivery Timer ◯
2.4 Truck or Kitchen? ◯
2.5 Kitchen Lab: ping ◯

3

RECIPE 03

3.1 Good Pup! ◯
3.2 Noodle Stall Gamble ◯
3.3 Pup Gridworld ◯
3.4 Reward Hacking ◯
3.5 Kitchen Lab: GridWorld ◯

4

RECIPE 04

4.1 Sensor Match ◯
4.2 Is the Steak Done? ◯
4.3 Kitchen Lab: phone sonar ◯

5

RECIPE 05

5.1 Fusion Saves Trips ◯
5.2 Does It Fit? ◯
5.3 Kitchen Manager ◯
5.4 Energy Bill ◯

6

MY RANK

0–10 Gold = Kitchen Helper
11–20 Gold = Line Cook
21+ Gold = HEAD CHEF!

Total Gold: ____

Meet the Kitchen

You are the new apprentice. By the end, you run the whole restaurant chain!

MAMA

Your teacher. Cheers when you get it right, gets fire-eyes when something burns — then always helps you fix it.



THE KIDS

Ask the "but why?" questions a beginner would ask. If you wonder it, they say it.



PAPA

Loves shortcuts. When Papa says "Easy! Just...", watch out: it is a common mistake.



THE KITCHEN PUP

Learns tricks from treats. Star of Recipe 03 (Behaviourist AI).



RECIPE 01

The Kitchen Crew — AI Hardware Architecture





Mama's Problem: "1,000 plates of fried rice in one minute! Who should cook — one master chef, a thousand helpers, or a machine?"

Step 1 · What Does AI Cook?



Predict first: does a neural network "think" with (a) logic puzzles (b) one tiny maths move, billions of times (c) internet searches?

The answer is (b): the MAC

Almost every layer multiplies inputs by learned weights and adds them up.

One "multiply, then add to the running total" = a MAC (multiply-accumulate). 🍳 Like adding one ingredient × its amount to the wok.

Scale it up: a layer with 1,024 inputs and 4,096 outputs = 1,024 × 4,096 = 4,194,304 MACs — for ONE input. Big language models need billions to trillions per answer.



Worked example: one neuron

weights w = [0.5, −1.0, 2.0] inputs x = [4, 3, 1]

total = 0
+ 0.5 × 4 = 2.0
+ (−1.0 × 3) = −1.0
+ 2.0 × 1 = 1.0 → output 1.0 (3 MACs)

🔬 Real tech: 1 MAC = 2 FLOPs (one multiply + one add). A matrix groups the weights so hardware can do many MACs at once; a tensor is a many-dimensional matrix — the "T" in TPU.



Mini-game 1.1 · Stir-Fry Maths

Cook a neuron by hand, then count the work in a whole layer.



🎯 Goal

Compute a neuron output with MACs and count the work in a layer.

🧺 Prep

Pen, paper, calculator. The worked example on the previous page.

🔪 Task

(1) w = [2, −1, 0.5], x = [1, 4, 6]: write each MAC step and the output.
(2) A layer has 512 inputs and 2,048 outputs. How many MACs per input? How many FLOPs?

👀 Observe

Write your running total after every single step.

💭 Reflect

Why would a chip that does MACs very fast speed up almost the whole model?

⭐ Mama's Feedback

(1) 0 → 2 → −2 → 1.0 (3 MACs).
(2) 512 × 2,048 = 1,048,576 MACs = 2,097,152 FLOPs.
Reflect: MACs are nearly all of a network's work, so speeding them up speeds up nearly everything.

Step 1b · Writing the Recipe vs Cooking the Order



Training and inference use the same model in two very different ways.

📖 Training = writing the recipe

Adjust the model's weights using lots of data.
- Rare, huge jobs (days to months) in data centres
- Higher precision (FP32 / BF16 / FP16)
- Big batches

🍽️ Inference = cooking each order
Use the finished weights to answer a request.
- Millions of times a day
- Latency, energy and cost per request matter
- Often batch size 1 and low precision (INT8, 4-bit)





Taste Test

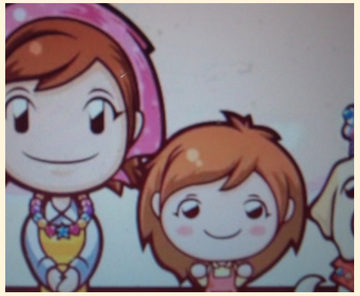
Training or inference?
(a) Face ID unlocking a phone
(b) fine-tuning a model on 100 GPUs for 3 weeks
(c) a chatbot answering 10,000 requests per second
→ Answers: (a) inference (b) training (c) inference

🔬 Precision = bits per number: FP32 = 4 bytes, FP16 = 2 bytes, INT8 = 1 byte. Fewer bits → less memory and energy, a little less accuracy.

Papa says: "Easy! The quoted peak speed is what I'll get." ✗ Peak = every unit busy every cycle. Real programs reach much less (utilisation).



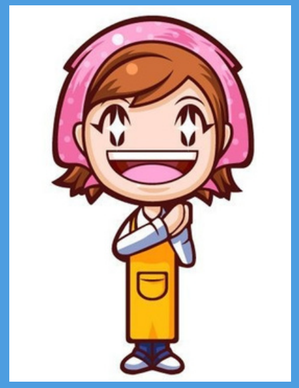
Step 2 · Meet the Crew



Rule of the kitchen: the more specialised the cook, the more efficient at that dish — and the less useful for anything else.

CPU · THE HEAD CHEF

A few very skilled cores that can cook ANY dish, one step at a time each. Flexible, low latency. Best for mixed tasks, data loading and small models.



GPU · 1,000 LINE COOKS

Thousands of simple cores all doing "chop these onions" together. High parallel throughput. Best for training and serving large models.



TPU · THE DUMPLING CONVEYOR

Google's chip built around a grid of MAC units. Highest efficiency for neural-network maths, but the least flexible.



NPU · THE HOME RICE COOKER

A small, low-power AI engine inside a phone or laptop chip (Apple Neural Engine, Qualcomm Hexagon). 🔬 CPUs spend chip area on caches and control; GPUs on many arithmetic units; TPUs on the MAC grid.



Mini-game 1.2 · Hire the Chef

Four restaurants need a cook. Pick the right chip and defend your choice.



🎯 Goal

Match a chip to a workload and justify it.

🧺 Prep

The crew cards on the previous page.

🔪 Task

Choose CPU, GPU, TPU or NPU:
(a) web server running mixed tasks
(b) training a large image model
(c) fixed, high-volume inference service in a data centre
(d) voice assistant in a battery smart speaker — rank NPU, CPU, GPU

👀 Observe

For each, write the ONE feature that decided it (flexibility, parallel throughput, efficiency per watt …).

💭 Reflect

Why might a start-up still choose a GPU even though the TPU was faster in Google's tests?

⭐ Mama's Feedback

(a) CPU — flexibility. (b) GPU (or TPU on Google Cloud). (c) TPU / fixed accelerator — best per watt. (d) NPU > CPU > GPU — a GPU drains a small battery.
Reflect: unsupported ops fall back to the CPU; team already uses GPU software; availability/cost; memory-bound jobs gain little.

Step 2b · The Dumpling Conveyor (Systolic Array)

Pass the dumplings hand-to-hand instead of running back to the fridge each time.



How it works

A systolic array is a grid of MAC cells. Each cell keeps ONE weight. Inputs and partial sums are passed to the neighbour — like dumplings on a conveyor — instead of being fetched from memory again and again.

🔬 First TPU (Jouppi et al., 2017): 65,536 8-bit MACs, 92 TOPS peak, ~15–30× faster and ~30–80× more TOPS per watt than 2015 CPUs/GPUs — but 4 of 6 workloads were limited by memory bandwidth. A historical comparison, not today's ranking.



Worked example: a 2 × 2 conveyor

W = [[1, 2], [3, 4]], x = [5, 6]

Cycle 1: x1 = 5 enters column 1
→ 1 × 5 = 5 and 3 × 5 = 15

Cycle 2: x2 = 6 reaches column 2
→ 2 × 6 + 5 = 17 and 4 × 6 + 15 = 39

Result y = [17, 39] with 4 MACs.

Memory reads: systolic = 4 weights + 2 inputs = 6; without passing = 8. The saving grows with array size.

Mini-game 1.3 · Conveyor Belt Trace

Be the conveyor belt: trace a 2 × 2 systolic array cycle by cycle.



🎯 Goal

Trace a weight-stationary systolic array cycle by cycle.



🧺 Prep

The worked example on the previous page, pen and paper.



🔪 Task

W = [[2, 1], [0, 3]], x = [4, 5].
Cycle 1: row-1 sum = __, row-2 sum = __
Cycle 2: row-1 sum = __, row-2 sum = __
Count memory reads with and without neighbour passing.

👀 Observe

Which numbers were read from memory, and which were handed along?



💭 Reflect

Why does reusing data matter more than adding more MAC cells?

⭐ Mama's Feedback

Cycle 1: 2×4 = 8 and 0×4 = 0. Cycle 2: 1×5 + 8 = 13 and 3×5 + 0 = 15 → y = [13, 15].
Reads: 6 (systolic) vs 8 (no reuse).
Reflect: fetching data costs far more energy and time than multiplying it (Step 4).

Step 3 · Cooking in Parallel

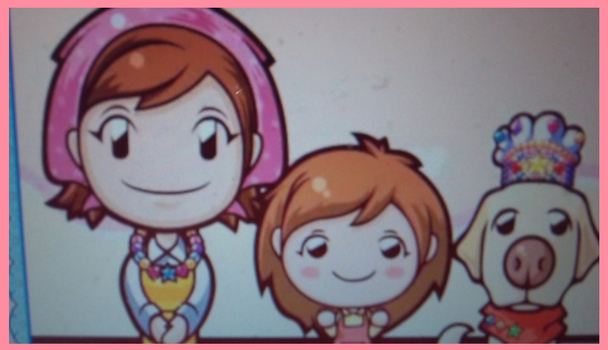


Mama's Problem: one chef chops 4,194,304 onions one by one; 4,096 helpers chop 4,096 at a time. How much faster?

The race

- Idealised CPU core: 1 MAC per cycle at 3 GHz → $4{,}194{,}304 \div 3\times10^{9} \approx 1.4$ ms
- Idealised accelerator: 4,096 MAC units at 1 GHz → $4{,}194{,}304 \div 4.096\times10^{12} \approx 1.0$ µs
- ≈ 1,365× faster, even with a slower clock — it does 4,096 MACs per cycle.

Four levels of parallelism: instruction-level (one core overlaps steps) · data-level SIMD/SIMT (GPU warps) · matrix-level (systolic arrays) · across chips (Recipes 02 & 05).



But… four things stop the speed-up

1. Amdahl's law — the part that can't be shared caps everything
2. Idle units — not enough independent MACs
3. Feeding the units — memory can't keep up
4. Communication — chips swapping results

🔬 Other boosters: quantisation (smaller numbers), specialised units (Tensor Cores, Neural Engine), batching (reuse weights).



Mini-game 1.4 · Chopping Race (Amdahl's Law)



Add more cooks… and discover the one wok that slows everyone down.

🎯 Goal

Discover why adding cooks stops helping.



🧺 Prep

speed-up = 1 ÷ ((1 − p) + p ÷ N)
p = share of work that can be split; N = number of cooks. Calculator + graph paper.

🔪 Task

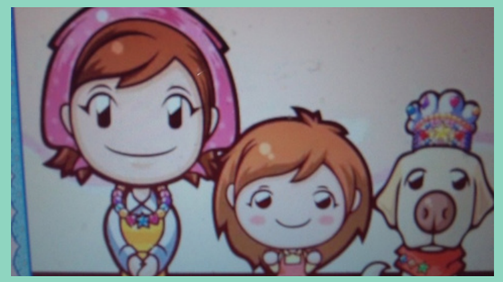
An order is 90% chopping (p = 0.9) and 10% waiting for the one wok. PREDICT the speed-up with 100 cooks. Then fill a table for N = 1, 2, 4, 8, 16, 100, ∞ and plot it.

👀 Observe

Check your table: N=1 → 1.0× · 2 → 1.8× · 4 → 3.1× · 8 → 4.7× · 16 → 6.4× · 100 → 9.2× · ∞ → 10×. Where does the curve flatten?

💭 Reflect

A job is 80% parallel. Speed-up on 8 units? Limit with unlimited units?



⭐ Mama's Feedback

1 ÷ (0.2 + 0.8 ÷ 8) = 1 ÷ 0.3 ≈ 3.3×; limit 1 ÷ 0.2 = 5×.
The serial 20% (the single wok!) caps everything. In AI pipelines the "wok" is data loading, pre-processing or steps that cannot be split.

Step 4 · The Pantry Problem (Memory Hierarchy)



Moving food costs more than cooking it. Keep the ingredients on the chopping board!

From hands to supermarket

✋ In your hands = registers (tiny, instant)
🔪 Chopping board = cache / on-chip SRAM
🧊 Fridge across the kitchen = DRAM / HBM
🛒 Supermarket = storage (huge, slow)

Faster memory is smaller and more expensive; bigger memory is slower and cheaper. A good cook plans so everything needed is already on the board — good AI hardware and software do the same.





The surprising fact (Horowitz, 2014)

45 nm chip, energy per operation:
- 32-bit multiply ≈ 3.7 pJ
- Read from small SRAM ≈ 5 pJ
- Read from DRAM ≈ 640 pJ

→ A DRAM read costs ≈ 173× a multiply and ≈ 128× an SRAM read.

🔬 HBM = DRAM stacked right next to the chip for very high bandwidth. Unified memory (Apple silicon) = CPU, GPU and NPU share one pool, so nothing is copied.

Mini-game 1.5 · Pantry Run

How far is each ingredient — and how big is the shopping bag?



🎯 Goal

Feel why data movement and number size matter.



🧺 Prep

The energy table on the previous page; 1 byte = 8 bits.



🔪 Task

(1) Rank, cheapest first: DRAM read, register access, small SRAM read.
(2) A 7-billion-parameter model: memory for its weights in FP16? INT8? 4-bit?
(3) A chip doubles its MAC units but keeps the same bandwidth. Effect on a memory-bound job?

👀 Observe

Compare your answers to a phone's memory (≈ 8 GB, shared with other apps).



💭 Reflect

Which helps a memory-bound workload more: more MACs, or smaller weights?

⭐ Mama's Feedback

(1) register < SRAM (~5 pJ) < DRAM (~640 pJ).
(2) 14 GB · 7 GB · 3.5 GB.
(3) Almost nothing — the units just wait for data.
Reflect: smaller weights (fewer bytes to move).

Step 4b · Cook Speed vs Delivery Speed (Roofline)



Is the cook too slow — or are the groceries arriving too slowly?

The roofline model

Arithmetic intensity = FLOPs ÷ bytes moved (how much cooking per bag of groceries).

Attainable = min(peak compute, intensity × memory bandwidth). Plotted, it looks like a roof: a slope (memory limit) meeting a flat top (compute limit).

Ridge point = peak ÷ bandwidth. Below it: memory-bound (cook waits). Above it: compute-bound (cook is busy).





Worked example: one layer

Chip: 10 TFLOP/s, 500 GB/s → ridge 20 FLOP/byte. Layer 1,024 × 4,096 in FP16:

- Batch 1: intensity ≈ 1.0 → 0.5 TFLOP/s = 5% of peak (memory-bound; ~95% of the time waiting)
- Batch 32: intensity ≈ 30.8 → 10 TFLOP/s = 100% (compute-bound)

🍱 Batching = cook 32 plates from one grocery delivery: weights fetched once, used 32 times.
⚠ But each order may wait longer (latency). Phones answer one user at a time → memory-bound.

Mini-game 1.6 · Batch Cooking

Find the batch size where the kitchen stops waiting for groceries.



🎯 Goal

Use the roofline to decide memory-bound vs compute-bound.



🧺 Prep

Same chip (10 TFLOP/s, 500 GB/s, ridge 20).
Layer: 512 inputs × 2,048 outputs, FP16.
Weights fetched once per batch; inputs/outputs once per sample.

🔪 Task

(1) weights and weight bytes (2) FLOPs at batch 1 (3) intensity at batch 1, 8, 32 — memory- or compute-bound? (4) attainable TFLOP/s and % of peak (5) smallest compute-bound batch

👀 Observe

Record a table: batch | intensity | bound | attainable | % peak.



💭 Reflect

A voice assistant answers one user at a time within milliseconds. What hardware would you choose?

⭐ Mama's Feedback

(1) 1,048,576 weights = 2,097,152 bytes (2) 2,097,152 FLOPs (3–4) b1 ≈ 1.0 memory 0.5 TF 5% · b8 ≈ 7.9 memory 3.9 TF 39% · b32 ≈ 29.7 compute 10 TF 100% (5) batch 22.
Reflect: batch 1 is memory-bound → high bandwidth, on-chip memory, low-precision weights (an NPU).

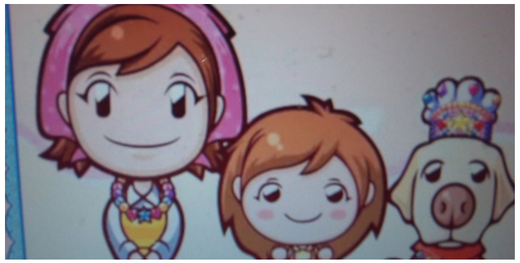
Step 5 · Case: Apple Neural Engine



The home rice cooker 🍚 Training happens in the big data-centre kitchen; the finished recipe is cooked at home, on your phone.

What is it?

The NPU inside Apple's chips. First shipped in 2017 in the A11 Bionic (iPhone X), described as 600 billion operations per second. It shares one system-on-a-chip and unified memory with the CPU and GPU; Core ML decides where each part of a model runs.



Face ID: sense → decide → act

1 Scan (TrueDepth camera)
2 Turn the face into numbers
3 AI inference on the Neural Engine
4 Compare with your saved face
5 Unlock
Face data stays on the device (Secure Enclave).



Pros & cons

✅ fast · energy-efficient · private · real-time · works offline

❌ limited compute and memory · not every operation is supported (falls back to the CPU) · big models still need the cloud



Why it fits Recipe 01

Face ID runs at batch size 1 → memory-bound. So the Neural Engine relies on on-chip memory, unified memory and low-precision (INT8/FP16) weights — Steps 1–4 in one real device.

Taste Test 01 🍽️

Choose A, B or C for each dish, then turn the page to see if Mama is proud.



Q1 🍙

Face ID processes one face at a time. On the roofline it is most likely…

A) compute-bound
B) memory-bound
C) network-bound



Q2 🍜

A 3-billion-parameter model quantised to 4-bit needs about…

A) 12 GB
B) 6 GB
C) 1.5 GB



Q3 🥟

One unsupported operation sits in the middle of a model. Why can it make the NPU slow?

A) the NPU overheats
B) data must move to the CPU and back
C) Core ML deletes the operation



Scoring

Write your answers here: Q1 __ Q2 __ Q3 __ → answers on the next page. All right = 🥇 Gold.

Taste Test 01 · Answers



All right? Mama is proud! Missed one? Mama will fix it — re-read Steps 4–5 and try again for Silver.

Q1 = B

Batch 1 means low arithmetic intensity: the weights are fetched for just one input, so memory is the limit.



Q2 = C

$3 \times 10^9 \times 0.5$ bytes = 1.5 GB. It fits on a phone with 8 GB of shared memory — but it is still a big share.



Q3 = B

Each switch to the CPU and back moves data between processors and loses the accelerator's efficiency. The data movement, not the operation, is the cost.



Mama says

"Delicious! Stamp your Score Card, then serve the Recipe Card on the next page."





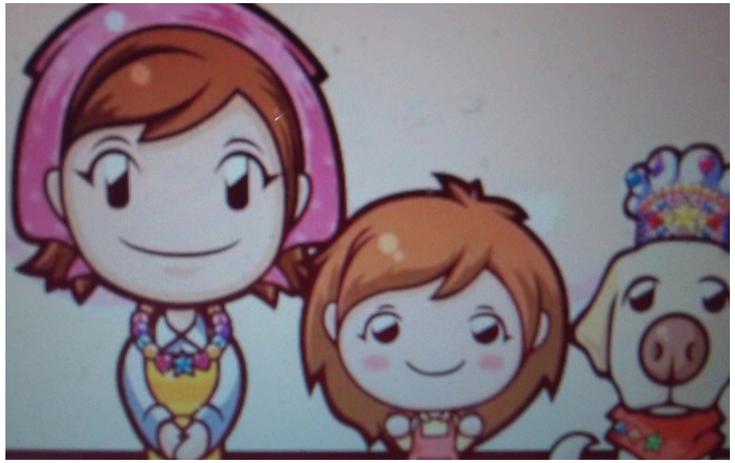
Recipe Card 01 · The Kitchen Crew



Cut out and keep: everything from Recipe 01 on one card.

METHOD (big ideas)

1. AI is mostly MACs — speed those up and you speed up the model.
2. Specialised chips trade flexibility for efficiency (CPU → GPU → TPU/NPU).
3. Parallelism helps until the serial part, idle units, memory or communication get in the way (Amdahl).
4. Moving data costs more than computing on it — keep ingredients on the chopping board (memory hierarchy, roofline, batching).
5. On-device AI (NPU) wins on privacy, latency and battery, but is limited by memory and supported operations.





INGREDIENTS (key terms)

MAC · FLOP · training / inference · CPU / GPU / TPU / NPU · systolic array · Amdahl's law · memory hierarchy · arithmetic intensity · roofline · batching · quantisation

🔓 Unlocked: Recipe Book page 1!





